

Git-Flows

Novice

Weight: 1

Project will start Aug 4, 2025 12:00 AM, must end by Aug 11, 2025 12:00 AM

Checker was released at Aug 4, 2025 12:00 AM

Manual QA review must be done (request it when you are done with the project)

An auto review will be launched at the deadline

Overview

Git-Flow is a branching model for Git, proposed by Vincent Driessen, that helps developers manage features, releases, and hotfixes in a consistent and scalable way. It introduces well-defined roles for branches and helps teams coordinate code changes more effectively. Git-Flow is particularly beneficial for large-scale projects where multiple developers work simultaneously on various aspects of the codebase.

Git-Flow revolves around **five primary branch types**:

- **main** (or **master**) – production-ready code
- **develop** – ongoing development
- **feature/*** – new features in progress
- **release/*** – preparation for production releases
- **hotfix/*** – critical bug fixes on the main branch

Relevance in the Development Process

Git-Flow improves:

- **Code organization** by clearly separating development stages
- **Team collaboration** through structured branching and merging workflows
- **Code stability** by allowing features to be tested and integrated before reaching production
- **Release management** by isolating release-specific tasks

Git-Flow is widely adopted in **agile and CI/CD** environments, ensuring seamless integration and deployment pipelines while reducing conflicts and regression bugs.

Learning Objectives

By the end of this, learners should be able to:

- Understand the purpose and structure of Git-Flow.
- Identify the different branch types and their roles.
- Apply Git-Flow in real-world collaborative development projects.
- Manage feature development, hotfixes, and release cycles using Git best practices.

Learning Outcomes

Learners will be able to:

- Explain how Git-Flow helps manage large codebases and team contributions.
- Create and manage branches following the Git-Flow model.
- Use Git commands to initiate features, releases, and hotfix branches.
- Integrate Git-Flow into CI/CD pipelines for automated testing and deployments.

Git-Flow Best Practices

Best Practice	Description
Start with <code>develop</code>	Always branch off from <code>develop</code> for new features, not <code>main</code> .
Feature Isolation	Keep each feature in its own branch to reduce merge conflicts.
Merge via Pull Requests	Use pull/merge requests for all merges to ensure code review.
Keep <code>main</code> clean	Only production-ready code goes into <code>main</code> . Never push untested code here.
Tag Releases	Use Git tags on the <code>main</code> branch to mark official release points.
Use <code>hotfix/*</code> for urgent bugs	Apply emergency fixes directly to <code>main</code> via a <code>hotfix/</code> branch and merge them back into <code>develop</code> .
Document your workflow	Maintain clear documentation of your Git-Flow in your project's README or internal wiki.

Common Git-Flow Commands

```
git flow init                # Initialize Git-Flow
git flow feature start <name> # Start a new feature branch
git flow feature finish <name> # Finish feature and merge into develop
git flow release start <x.x.x> # Start a release branch
git flow release finish <x.x.x> # Merge release into main and develop
git flow hotfix start <x.x.x> # Start a hotfix branch
git flow hotfix finish <x.x.x> # Finish hotfix and merge into main & develop
```

Quiz questions

Great! You've completed the quiz successfully! Keep going! ([Show quiz](#))

Tasks

0. Setting up gitflow

mandatory

Objective: Understand and initialize a GitFlow workflow

Instructions:

- Install git-flow if not already installed. Steps [here](#)
- Create an empty repository `ALXprodev-advanced_git`
- Clone your repository to have it available locally.
- Create a branch `develop`
- Push the develop branch to the remote repository
- Initialize Git Flow within the repository using default settings i.e `git-flow init [-d]`
- Create an empty `README.md` file
- Commit your file to staging and Push

Repo:

- GitHub repository: `ALXprodev-advanced_git`
- File: `README.md`

1. Creating a feature branch

mandatory

Objective: Learn how to work with feature branches in GitFlow.

Instructions:

- Create a new feature branch `feature/implement-login` from the `develop` branch
- In the `feature/implement-login` create a new directory called `login-page`, inside it create a `README.md` file with the message: `Login Feature Coming soon`
- Commit your changes and push to github
- Use commit message `feat: scaffolding login page`

Repo:

- GitHub repository: `ALXprodev-advanced_git`
- Directory: `login-page`
- File: `README.md`

2. Creating a Release Branch

mandatory

Objective: Understand the release process in GitFlow.

Instructions:

- Create a new feature branch `feature/implement-signup` from the `develop` branch and create a directory called `signup-page` with a `README.md` file with the message `feature coming soon`
- Commit the changes and push to github
- Merge the 2 branches to the `develop` branch. Fix conflicts if any
- Create a new release branch `release/1.0.0`
- Make changes to `signup/README.md` file by adding the following text:” `data requirements: email, firstName, lastName, profilePic]`” inside the release branch
- Commit and push changes to github then merge the `release` branch to the `main` branch and the `develop` branch
- Tag the release with `v1.0.0` and push the tags to the remote repository

Repo:

- GitHub repository: `ALXprodev-advanced_git`
- Directory: `login-page, signup-page`
- File: ``login-page/README.md, signup-page/README.md``

3. Git Hooks and Automation

mandatory

Objective: Implement Git hooks to automate parts of the GitFlow process.

Instructions:

- Implement a pre-commit hook in the file `.git/hooks/pre-commit` that checks if each directory in a Git repository has a `README` file
- Implement a post-merge hook that logs the merge after completing a merge into the main branch.

Hint: Read more [here](#)

Repo:

- GitHub repository: [ALXprodev-advanced_git](#)

4. Manual Review

mandatory

Repo:

- GitHub repository: [ALXprodev-advanced_git](#)